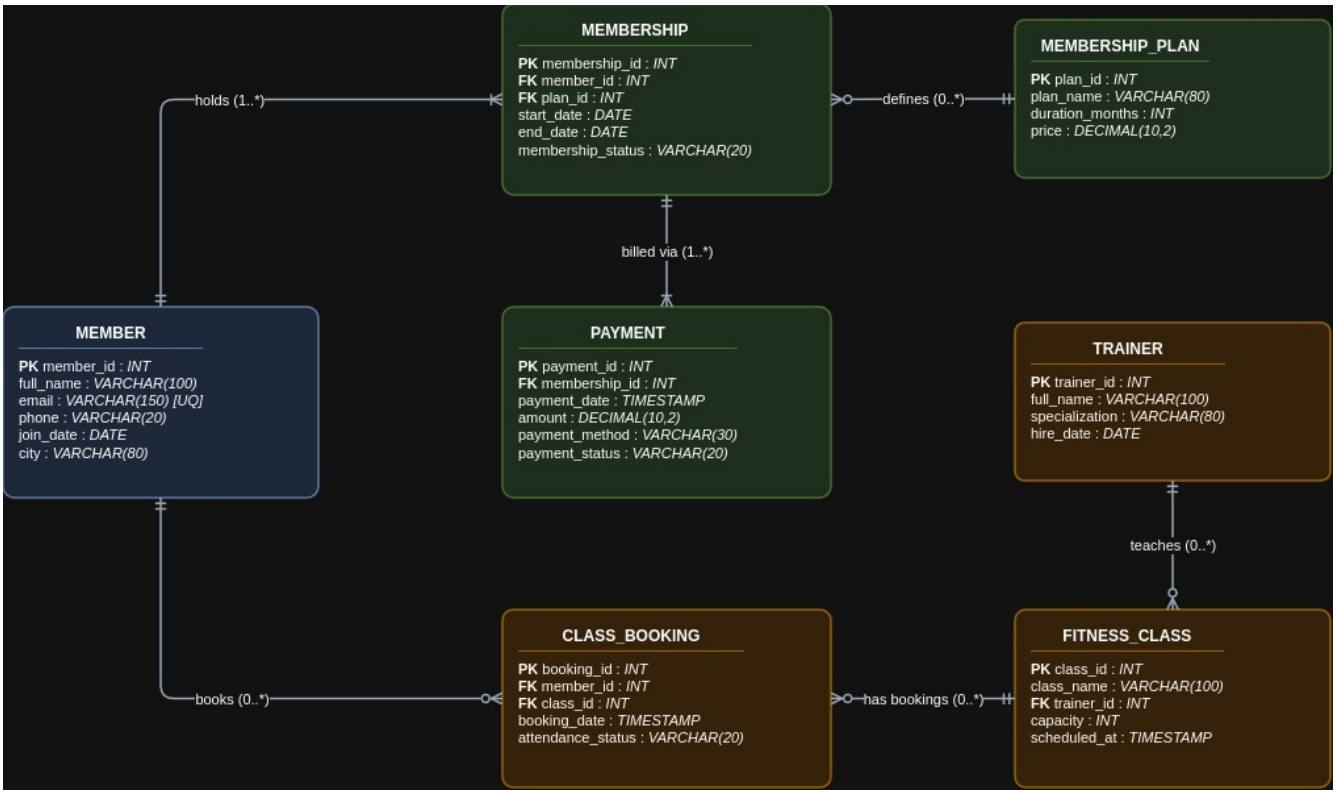
Part A - ERD Design



```
-- CREATE TABLE statements

CREATE TABLE member (
    member_id    NUMBER PRIMARY
    full_name    VARCHAR2(100) NOT
NULL,     email
    phone
    join_date    DATE NOT
                VARCHAR2(50) NOT NULL
city );

CREATE TABLE membership_plan (
    plan_id          NUMBER PRIMARY KEY,
    plan_name        VARCHAR2(50) NOT
    duration_months  NUMBER(3) NOT NULL,
    price            NUMBER(10,2) NOT NULL,
    CONSTRAINT chk_plan_duration CHECK (duration_months >
0),    CONSTRAINT chk_plan_price    CHECK (price > 0)
);

CREATE TABLE trainer (
    trainer_id      NUMBER PRIMARY KEY,
    full_name           VARCHAR2(100) NOT
NULL,     specialization  VARCHAR2(50),
    hire_date           DATE
NOT NULL );

CREATE TABLE membership (
    membership_id      NUMBER PRIMARY
member_id              KEY, NUMBER NOT
    plan_id            NULL, NUMBER NOT
    start_date                DATE
NOT NULL,     end_date
    membership_status  VARCHAR2(20) NOT NULL,
```

```
        REFERENCES member(member_id),
    CONSTRAINT fk_membership_plan  FOREIGN KEY (plan_id)
        REFERENCES membership_plan(plan_id),
    CONSTRAINT chk_membership_status CHECK (membership_status IN ('
);

CREATE TABLE fitness_class (
    class_id      NUMBER PRIMARY KEY,
    class_name    VARCHAR2(50) NOT NULL,
    trainer_id    NUMBER,
    capacity      NUMBER(4) NOT NULL,
    scheduled_at  TIMESTAMP NOT NULL,
    CONSTRAINT fk_class_trainer FOREIGN KEY (trainer_id)
        REFERENCES trainer(trainer_id),
    CONSTRAINT chk_class_capacity CHECK (capacity > 0)
);

CREATE TABLE class_booking (
    booking_id         NUMBER PRIMARY KEY,
    member_id          NUMBER NOT NULL,
    class_id           NUMBER NOT NULL,
    booking_date       DATE NOT NULL,
    attendance_status  VARCHAR2(20) NOT NULL,
    CONSTRAINT fk_booking_member FOREIGN KEY (member_id)
        REFERENCES member(member_id),
    CONSTRAINT fk_booking_class  FOREIGN KEY (class_id)
        REFERENCES fitness_class(class_id),
    CONSTRAINT chk_attendance_status CHECK (attendance_status IN ('
);

CREATE TABLE payment (
    payment_id      NUMBER PRIMARY KEY,
    membership_id   NUMBER NOT NULL,
    payment_date    DATE NOT NULL,
```

```
    payment_method  VARCHAR2(20) NOT NULL,
    payment_status  VARCHAR2(20) NOT NULL,
    CONSTRAINT fk_payment_membership FOREIGN KEY (membership_id)
        REFERENCES membership(membership_id),
    CONSTRAINT chk_payment_amount  CHECK (amount > 0),
    CONSTRAINT chk_payment_method  CHECK (payment_method IN ('Cash'
    CONSTRAINT chk_payment_status  CHECK (payment_status IN ('Succe
);

-- INSERT statements

-- MEMBER
INSERT INTO member VALUES (1, 'Alice Johnson', 'alice@example.com',
INSERT INTO member VALUES (2, 'Bob Smith', 'bob@example.com', '0102
INSERT INTO member VALUES (3, 'Charlie Brown', 'charlie@example.cor
INSERT INTO member VALUES (4, 'Diana Prince', 'diana@example.com',
INSERT INTO member VALUES (5, 'Evan Wright', 'evan@example.com', '0
INSERT INTO member VALUES (6, 'Fiona Green', 'fiona@example.com', '
INSERT INTO member VALUES (7, 'George Hall', 'george@example.com',
INSERT INTO member VALUES (8, 'Hannah Lee', 'hannah@example.com', '
INSERT INTO member VALUES (9, 'Ian Clark', 'ian@example.com', '0109
INSERT INTO member VALUES (10, 'Julia Adams', 'julia@example.com',
INSERT INTO member VALUES (11, 'Kevin White', 'kevin@example.com',
INSERT INTO member VALUES (12, 'Laura Black', 'laura@example.com',
INSERT INTO member VALUES (13, 'No Contact', NULL, NULL, DATE '2024

-- MEMBERSHIP_PLAN
INSERT INTO membership_plan VALUES (1, 'Basic Monthly', 1, 500.00);
INSERT INTO membership_plan VALUES (2, 'Standard Quarterly', 3, 120
INSERT INTO membership_plan VALUES (3, 'Premium Yearly', 12, 5000.0
INSERT INTO membership_plan VALUES (4, 'Student Plan', 6, 800.00);
INSERT INTO membership_plan VALUES (5, 'Day Pass Pack', 1, 200.00);

-- TRAINER
```

```
INSERT INTO trainer VALUES (2, 'Sarah Coach', 'Yoga', DATE '2021-03
INSERT INTO trainer VALUES (3, 'Mike Fitness', 'Cardio', DATE '2019
INSERT INTO trainer VALUES (4, 'Linda Wellness', 'Pilates', DATE '2
INSERT INTO trainer VALUES (5, 'Tom Power', 'CrossFit', DATE '2021-
INSERT INTO trainer VALUES (6, 'Nina Zen', 'Meditation', DATE '2023

-- MEMBERSHIP
INSERT INTO membership VALUES (1, 1, 1, DATE '2024-01-01', DATE '20
INSERT INTO membership VALUES (2, 1, 2, DATE '2024-02-01', DATE '20
INSERT INTO membership VALUES (3, 2, 1, DATE '2024-01-15', DATE '20
INSERT INTO membership VALUES (4, 3, 3, DATE '2024-01-01', DATE '20
INSERT INTO membership VALUES (5, 4, 4, DATE '2024-03-01', DATE '20
INSERT INTO membership VALUES (6, 5, 1, DATE '2024-04-01', DATE '20
INSERT INTO membership VALUES (7, 6, 2, DATE '2024-05-01', DATE '20
INSERT INTO membership VALUES (8, 8, 3, DATE '2024-06-01', DATE '20
INSERT INTO membership VALUES (9, 9, 4, DATE '2024-07-01', DATE '20
INSERT INTO membership VALUES (10, 10, 1, DATE '2024-08-01', DATE '

-- FITNESS_CLASS
INSERT INTO fitness_class VALUES (1, 'Morning Yoga', 2, 20, TIMESTA
INSERT INTO fitness_class VALUES (2, 'Strength Training', 1, 15, TI
INSERT INTO fitness_class VALUES (3, 'Cardio Blast', 3, 25, TIMESTA
INSERT INTO fitness_class VALUES (4, 'Pilates Core', 4, 12, TIMESTA
INSERT INTO fitness_class VALUES (5, 'CrossFit WOD', 5, 18, TIMESTA
INSERT INTO fitness_class VALUES (6, 'HIIT Session', 1, 20, TIMESTA
INSERT INTO fitness_class VALUES (7, 'Meditation Class', NULL, 15,
INSERT INTO fitness_class VALUES (8, 'Zumba Dance', 2, 30, TIMESTAM

-- CLASS_BOOKING
INSERT INTO class_booking VALUES (1, 1, 1, DATE '2024-10-01', 'Book
INSERT INTO class_booking VALUES (2, 1, 2, DATE '2024-10-02', 'Atte
INSERT INTO class_booking VALUES (3, 2, 1, DATE '2024-10-01', 'Atte
INSERT INTO class_booking VALUES (4, 3, 3, DATE '2024-10-03', 'Book
INSERT INTO class_booking VALUES (5, 4, 4, DATE '2024-10-04', 'Abse
```

```
INSERT INTO class_booking VALUES (7, 6, 6, DATE '2024-10-06', 'Book
INSERT INTO class_booking VALUES (8, 8, 3, DATE '2024-10-03', 'Atte
INSERT INTO class_booking VALUES (9, 9, 1, DATE '2024-10-01', 'Book
INSERT INTO class_booking VALUES (10, 10, 2, DATE '2024-10-02', 'At

-- PAYMENT
INSERT INTO payment VALUES (1, 1, DATE '2024-01-02', 500.00, 'Cash'
INSERT INTO payment VALUES (2, 2, DATE '2024-02-02', 1200.00, 'Card
INSERT INTO payment VALUES (3, 3, DATE '2024-01-20', 500.00, 'Onlin
INSERT INTO payment VALUES (4, 4, DATE '2024-01-05', 5000.00, 'Card
INSERT INTO payment VALUES (5, 5, DATE '2024-03-05', 800.00, 'Cash'
INSERT INTO payment VALUES (6, 6, DATE '2024-04-05', 500.00, 'Onlin
INSERT INTO payment VALUES (7, 7, DATE '2024-05-05', 1200.00, 'Card
INSERT INTO payment VALUES (8, 8, DATE '2024-06-05', 5000.00, 'Cash
INSERT INTO payment VALUES (9, 8, DATE '2024-07-05', 5000.00, 'Onli
INSERT INTO payment VALUES (10, 10, DATE '2024-08-05', 500.00, 'Cas

COMMIT;
```

```sql
-- 1. Membership details
SELECT m.membership_id,
       mem.full_name AS member_name,
       p.plan_name,
       m.start_date,
       m.membership_statu
FROM membership m
INNER JOIN member mem ON m.member_id = mem.member_id
INNER JOIN membership_plan p ON m.plan_id =

-- 2. Members and memberships
SELECT mem.member_id,
       mem.full_name,
       m.membership_id,
       m.start_date,
       m.membership_statu
FROM member mem
LEFT JOIN membership m ON mem.member_id = m.member_id;

-- 3. Classes and bookings (RIGHT JOIN)
SELECT fc.class_id,
       fc.class_name,
cb.booking_id,
cb.member_id,
       cb.booking_date,
       cb.attendance_status
FROM class_booking cb
RIGHT JOIN fitness_class fc ON cb.class_id = fc.class_id;

-- 4. Trainers and classes
SELECT t.trainer_id,
       t.full_name AS
trainer_name,       fc.class_id,
       fc.class_name
```

```
LEFT JOIN fitness_class fc ON t.trainer_id = fc.trainer_id;

-- 5. Plans summary
SELECT p.plan_name,
       COUNT(m.membership_id) AS number_of_memberships,
       AVG(p.price) AS average_plan_price
FROM membership_plan p
JOIN membership m ON p.plan_id = m.plan_id
GROUP BY p.plan_id, p.plan_name
ORDER BY average_plan_price DESC;

-- 6. Member booking counts
SELECT mem.full_name,
       COUNT(cb.booking_id) AS total_bookings
FROM member mem
LEFT JOIN class_booking cb ON mem.member_id = cb.member_id
GROUP BY mem.member_id, mem.full_name
ORDER BY total_bookings DESC, mem.full_name ASC;

-- 7. Successful revenue by plan
SELECT p.plan_name,
       SUM(pay.amount) AS total_revenue
FROM payment pay
JOIN membership m ON pay.membership_id = m.membership_id
JOIN membership_plan p ON m.plan_id = p.plan_id
WHERE pay.payment_status = 'Successful'
GROUP BY p.plan_name
ORDER BY total_revenue DESC;

-- 8. Class booking counts
SELECT fc.class_name,
       COUNT(cb.booking_id) AS number_of_bookings
FROM fitness_class fc
LEFT JOIN class_booking cb ON fc.class_id = cb.class_id
```

```sql
ORDER BY number_of_bookings DESC;

-- 9. Membership status label
SELECT membership_id,
       CASE membership_status
         WHEN 'Active'  THEN 'Active Membership'
         WHEN 'Expired' THEN 'Expired Membership'
         WHEN 'Pending' THEN 'Pending Membership'
       END AS status_label
FROM membership
ORDER BY membership_id;

-- 10. Member contact value
SELECT full_name,
       NVL(phone, NVL(email, 'No contact details')) AS contact_valu
FROM member;

-- 11. Amount paid per membership
SELECT m.membership_id,
       NVL(SUM(p.amount), 0) AS total_paid
FROM membership m
LEFT JOIN payment p ON m.membership_id = p.membership_id
GROUP BY m.membership_id
ORDER BY m.membership_id;

-- 12. Trainer class totals
SELECT t.full_name,
       COUNT(fc.class_id) AS class_count
FROM trainer t
LEFT JOIN fitness_class fc ON t.trainer_id = fc.trainer_id
GROUP BY t.trainer_id, t.full_name
ORDER BY class_count DESC;

-- 13. Payment status label
```

```
       membership_id,
       payment_method,
       amount,
       payment_date,
       CASE payment_status
         WHEN 'Successful' THEN 'Completed'
         WHEN 'Pending'    THEN 'Pending'
         WHEN 'Failed'     THEN 'Not completed'
       END AS status_label
FROM payment
ORDER BY payment_date DESC;

-- 14. Payment statistics
SELECT payment_method,
       SUM(amount) AS total_amount,
       AVG(amount) AS average_amount,
       MIN(amount) AS minimum_amount,
       MAX(amount) AS maximum_amount,
       COUNT(*) AS number_of_payments
FROM payment
GROUP BY payment_method;

-- 15. Members with multiple successful payments
SELECT mem.full_name,
       COUNT(pay.payment_id) AS successful_payments,
       SUM(pay.amount) AS total_paid
FROM member mem
JOIN membership m ON mem.member_id = m.member_id
JOIN payment pay ON m.membership_id = pay.membership_id
WHERE pay.payment_status = 'Successful'
GROUP BY mem.member_id, mem.full_name
HAVING COUNT(pay.payment_id) >= 2
ORDER BY total_paid DESC;
```